

Optimizing Hyperparameters with Metaheuristic Approaches: A Comparative Study and Hybridization for Enhanced Performance

Daksh Gupta (2023A7PS0106H), Kabir Grewal (2023A7PS1102H),
Siddharth Bhatia (2023A7PS1106H), Neha Bhagwat (2023A7PS0067H)

GitHub: <https://github.com/Neha-Bhagwat/Multi-Algorithm-HPO>

Keywords: Genetic Algorithm, Differential Evolution, Particle Swarm Optimization, Meta-heuristics, Artificial Neural Network, Bayesian Optimization, HyperBand Optimization, PyHopper

1. Introduction

This is the project for the course BITS F464, Machine Learning. Our aim is to compare algorithms for the most efficient Hyper Parameter Optimization. We have used several algorithms as mentioned in our chosen paper *Comparative Evaluation of Metaheuristic Algorithms for Hyperparameter Selection in Short-Term Weather Forecasting* (<https://arxiv.org/pdf/2309.02600>) and added Bayesian Optimization (BO) in combination with one of the algorithm DE to get the smallest errors in the results.

In this project, we conduct a comparative study of several metaheuristic algorithms for use in hyperparameter optimization in short-term weather forecasting models. On the ECTA 2023 dataset, we compare the performance of each algorithm for the optimization of the hyperparameters of a single machine learning model. The overall objective of this study is to establish the most optimal optimization method that maximizes the accuracy of the forecast as well as computational efficiency.

2. Dataset Description

In this paper, we aim to accurately predict the temperature on a given day using various other meteorological

features such as wind speed, relative humidity and the like. We do so by implementing a multi-layer ANN (Artificial Neural Network) that is populated using a dataset, to accurately adjust weights and biases and perform a forward pass through the neural network to predict what the temperature will be on a random day with arbitrary values of said meteorological features.

The dataset used in our implementation of said paper will be the same as the one used in theirs (to make things simpler and to better draw comparisons and calculate improvement), namely "ECTA 2023 dataset", which is a dataset consisting of hourly weather data from Ottawa, collected over a ten-year span.

In short, the paper aims to accurately tune the values of "hyperparameters", which are values related to the neural network that in most problem statements are given, and we merely simulate forward passes and perform backpropagation to accurately update weights and biases. Here by fine tuning the hyperparameters using various algorithms we find the best possible values for these using which the neural network can produce results with the lowest possible error.

The hyperparameters that we aim to fine tune in this project are *Learning rate*, *number of neurons in hidden layer 1*, *number of neurons in hidden layer 2*, and *batch size*. In this project we try to do so by implementing various algorithms already mentioned in the paper that we aim to improve the results of, as well

as by finding novel approaches that may yet improve the efficiency of said algorithms. The algorithms mentioned in the paper are as follows:

1. Genetic Algorithm (GA)
2. Differential Evolution (DE)
3. Particle Swarm Optimisation (PSO)
4. PyHopper
5. BOHB

The first three of these five are from a class of Metaheuristic Algorithms, which are named as such because they mimic natural processes and human behaviour to produce a solution to a problem that is as optimal as possible. PyHopper is a black-box optimisation platform that streamlines the hyperparameter tuning process, presented in a research paper (<https://arxiv.org/pdf/2210.04728>). BOHB is a method to tune hyperparameters that utilises both Bayesian Optimisation and Hyperband. Each algorithm will be discussed in its respective section.

The novel approach that we seek to implement that, we see, results in marginal improvements over the best of the methods listed above is an algorithm coded by hybridising Differential Evolution (DE) and Bayesian Optimisation (BO).

Our model's performance is evaluated using MAE (Mean Average Error), which serves as the primary metric throughout each implementation of the respective algorithms. By comparing the MAE generated by each algorithm, we seek to demonstrate the advantages of using the approaches mentioned above and also to draw comparisons to find out which method works best to serve our purpose of optimising model accuracy and training efficiency.

3. Preprocessing

For each of the approaches mentioned earlier, we first load the dataset and split it into training and testing sets.

Our dataset is loaded using the pandas library and is then filtered to use six features, namely *Wind Speed*, *Relative Humidity*, *Pressure*, *Wind Chill*, *Visibility* and *Dewpoint Temperature*. We then clean up

our dataframe by using `dropna()` to remove those rows with null values.

We split the dataset into training and testing sets with an 80:20 split between the two. We then use a standard scaler to normalise the values. X is defined as the tuple of features, and Y as the target variable, which for our project is Temperature.

4. Constructing the NN

Our ANN (Artificial Neural Network) is made using the Keras API of the Tensorflow library. It consists of two hidden layers, both of which utilise the ReLU activation function, and an output layer that uses the linear activation function.

Our model is then compiled using the ADAM (Adaptive Moment Estimation) optimiser from Keras and mean square error. As mentioned earlier, we track Mean Absolute Error (MAE) as an evaluation metric.

5. METAHEURISTICS

5.1 Genetic Algorithm (GA)

To optimize the hyperparameters of a neural network model predicting temperature based on weather features, I employed a Genetic Algorithm (GA). This approach systematically explores the hyperparameter space to identify the best combination of values that minimizes the Mean Absolute Error (MAE). The implementation of the GA was designed to evolve a population of solutions over several generations, gradually improving the quality of the solutions.

5.1.1. Population Initialization

The genetic algorithm starts with the initial population, which consists of a set of randomly generated individuals. Each individual represents a possible configuration of the model's hyperparameters: learning rate (lr), neurons in hidden layer 1 (neurons1), neurons in hidden layer 2 (neurons2), and batch size

(batch_size). These parameters are bounded within predefined ranges:

- Learning Rate (lr): Between 0.0001 and 0.1
- Neurons in Layer 1 and 2: Between 16 and 128 neurons
- Batch Size: Between 8 and 64

The population is created using a random process where each individual in the population represents a unique combination of these hyperparameters. This random initialization provides diverse candidate solutions, which is essential for GA's ability to explore different configurations.

5.1.2. Evaluation and Fitness Calculation

Each individual in the population is evaluated by training the neural network model using the corresponding hyperparameters. The model is trained on the training data and then evaluated on the test data, using MAE as the evaluation metric. The fitness of an individual is determined by the negative MAE; since GA aims to maximize fitness, a lower MAE leads to a higher fitness value.

The fitness evaluation is crucial in identifying which individuals contribute the most to the improvement of the model. During each iteration, the individuals with the lowest MAE (best fitness) are given a higher chance of reproduction.

5.1.3. Selection

The roulette wheel selection mechanism is used to choose individuals for reproduction. In this process, the probability of selecting an individual is proportional to its fitness. This method ensures that better-performing solutions have a higher chance of passing their genetic material to the next generation, simulating the concept of survival of the fittest.

5.1.4. Crossover and Mutation

Once parents are selected, they undergo crossover and mutation to create the next generation. After crossover, the child can undergo mutation, where

small, random changes are introduced to its genes. This step helps maintain genetic diversity within the population and prevents premature convergence by exploring new regions of the hyperparameter space. Mutation is performed with a probability of 20%.

5.1.5. Evolution and Termination

The algorithm runs for a predefined number of generations. In each generation, the new population replaces the old one, and the process of selection, crossover, and mutation is repeated. At the end of the GA process, the best individual (the combination of hyperparameters) is selected as the optimal configuration. This best solution is then used to train the final neural network model, which is evaluated to ensure it performs better than the initial configuration.

5.1.6. Observations

After running the GA algorithm, we got the final results:-

Best Learning Rate: **0.04225**

Best Number of Neurons in Layer 1: **19**

Best Number of Neurons in Layer 2: **19**

Best Batch Size: **43**

Best MAE: **0.0939**

5.2 Differential Evolution (DE)

5.2.1 Optimization Process

Differential Evolution (DE) was used to search for the best combination of hyperparameters. It began with a randomly generated population and improved candidate solutions over successive generations through mutation, crossover, and selection mechanisms.

5.2.2 Fitness Evaluation

Each candidate solution represented a specific hyperparameter set. Its fitness was evaluated by training the model on the training set and measuring

the MAE on test data. Candidates with better performance replaced weaker ones in the population.

5.2.3 Convergence Through Generations

Across multiple generations, the population gradually converged towards more optimal hyperparameters. The evolutionary nature of DE ensured that the search space was explored thoroughly, preventing early convergence on suboptimal results.

5.2.4 Results and Observations

After running the loop for DE and obtaining the results through 10 epochs, we get the following best parameters.

- Best learning rate: **0.02311**
- Best n1 (Number of neurons in hidden layer 1): **128**
- Best n2 (Number of neurons in hidden layer 2): **16**
- Batch size: **64**

We conclude by saying that the Mean Average Error (MAE) of the DE algorithm over ECTA dataset is **0.0723**

5.3 Particle Swarm Optimisation (PSO)

This part of the project pertains to the use of Particle Swarm Optimisation (PSO) to finely tune hyperparameters.

5.3.1 Initialisation of Variables

Prior to the main PSO loop we first initialise basic parameters - we set num_particles = 8 and num_iterations = 5. In addition to this, we set inertia to 0.5 and also c1 and c2 to 2.0 each. c1 and c2 are coefficients that determine how influenced a particle is by its previous vector, and how attracted it is to the global best, respectively. c1 = c2 is considered to be a balanced pair, and by Clerc's Constriction Coefficient we decide on the value of 2.0. Lower and upper bounds for each of the particles' attributes are defined as

[0.001,10,10,16] and [0.1,100,100,128] respectively, and then we define the following.

- a copy of the best current set
- the best score for each particle, with each particle initially thinking it's the best
- the global best position
- and the global best score.

5.3.2 The main PSO Loop

For each particle we define r1 and r2, random influence vectors. Then we update velocities and positions, while using np.clip() to ensure that these remain within bounds. We then evaluate the best scores, and should a score be the best (resulting in the least mean absolute error) we update the best score for each particle as well as the global best score (should it be better than that). This loop runs num_iterations times.

5.3.3 Results and Observations

Using the best values obtained from the main PSO Loop (gbest), we declare these to be the most optimal values of our model's hyperparameters and then make a forward pass through the neural network one last time using these values to obtain the MAE.

Using the dataset available and the values of parameters declared, we obtain the following data;

- Best learning rate: **0.03278**
- Best n1 (Number of neurons in hidden layer 1): **60**
- Best n2 (Number of neurons in hidden layer 2): **33**
- Batch size: **93**

And by using these values we obtain a Mean Average Error (MAE) of **0.1571**.

6. Other HPO Methods:

6.1 PyHopper

6.1.1 Introduction

This report presents the use of PyHopper for optimizing hyperparameters in a machine learning model designed to predict temperature based on weather-related features. The primary goal of the experiment is to explore how hyperparameter optimization can enhance the model's performance by minimizing the Mean Absolute Error (MAE).

6.1.2 Steps

The typical process of using PyHopper involves the following steps:

1. **Defining the search space:** A range of values for each hyperparameter is specified.
2. **Defining the objective function:** This function takes the hyperparameters as input, trains the model, and evaluates its performance, returning a metric (e.g., MAE) that PyHopper tries to minimize.
3. **Running the search:** PyHopper explores the search space and evaluates different combinations of hyperparameters to find the optimal configuration.

6.1.3 Experiment Setup

The experiment involves training a deep neural network model on weather-related features to predict temperature. The model's performance is evaluated using the MAE metric, and PyHopper is employed to find the optimal hyperparameters.

6.1.4 Search Space Definition

The hyperparameters to be optimized and their respective search spaces were defined as:

- **Learning Rate (lr):** Ranges from 0.0005 to 0.05.
- **Number of Neurons in the First Hidden Layer (n1):** Ranges from 10 to 100.
- **Number of Neurons in the Second Hidden Layer (n2):** Ranges from 10 to 100.
- **Batch Size (bs):** Ranges from 16 to 128.

6.1.5 Objective Function

The objective function evaluates the model's performance by training it with a given set of hyperparameters for 10 epochs and then measuring its MAE on the test set. The function returns the MAE, which PyHopper seeks to minimize.

6.1.6 Optimization Process

The optimization was set to run for 15 minutes to allow for a more thorough search through the hyperparameter space. By running the search for this duration, PyHopper was able to explore a range of configurations to identify the best combination of hyperparameters.

Observation and Conclusion

The hyperparameter optimization process using PyHopper yielded the following best configuration:

- Learning Rate (lr): **0.02594**
- Neurons in Layer 1 (n1): **100**
- Neurons in Layer 2 (n2): **31**
- Batch Size (bs): **18**

The final Mean Absolute Error (MAE) achieved was **0.1007**, highlighting significant improvement in predictive accuracy. PyHopper's approach to optimizing hyperparameters proved effective in fine-tuning the model, demonstrating its potential in enhancing model performance for weather prediction tasks. By automating the search for optimal hyperparameters, this method minimizes the time and effort required for manual tuning, making it a valuable

tool in machine learning tasks that require high accuracy and efficient model training.

7. Hybrid Differential Evolution + Bayesian Optimization (DE+BO)

7.1 Why Hybridise?

Having attained somewhat satisfactory results from it, we concluded that Differential Evolution (DE) is a strong metaheuristic for hyperparameter tuning in weather forecasting models. However DE alone has some notable limitations:

- It may require a large number of iterations to converge.
- It tends to perform weaker in fine-tuning or local exploitation.
- It has no built-in memory or model of the objective function, making it purely exploratory and sometimes inefficient in narrow search spaces.

Bayesian Optimization (BO), in contrast, is highly sample-efficient and excels at fine-tuning in promising areas by building a probabilistic model (typically a gaussian) of the objective function. However, BO can miss global optima if it starts in a poor region.

By combining DE and BO, we can get the best of both worlds - combining DE's global exploration to find good regions and BO's local exploitation to efficiently converge to the best solution. This hybrid DE+BO approach is especially suited for deep learning, where each evaluation is costly and the search space is large.

7.2 Implementation Overview

Framework Setup

The hybrid DE+BO optimization framework was implemented in Python using **Numpy** and **scikit-optimize (skopt)**. A simple feedforward neural network was constructed using **Keras** (TensorFlow

backend), with two hidden layers. The input features were generated using a sliding window approach on the time series data, and all values were normalized using **StandardScaler**.

Each model was trained for 10 epochs during optimization to allow fast convergence measurement, with the Mean Absolute Error (MAE) used as the evaluation metric.

The hybrid approach combined **Differential Evolution (DE)** with **Bayesian Optimization (BO)** in two sequential phases:

- **Phase 1 – Differential Evolution (DE):**
DE was run for 3 generations with a population size of 5. The mutation factor was set to 0.5 , and the crossover probability was 0.7, with an early stopping callback included to exit prematurely if no improvement was observed after several evaluations. The search space for DE was kept comparatively smaller to really test BO's capability of further optimization.
- **Phase 2 – Bayesian Optimization (BO):**
The best result from DE was passed as the initial point to **gp_minimize** from **scikit-optimize**, which then performed a more refined search using a Gaussian Process surrogate model. Around 30 optimization calls were made, and **n_random_starts** was set to 2 for additional exploration.

The best hyperparameters obtained from BO were then used to train the final model. This model was built with the same architecture but trained on the full dataset for more epochs (50), with early stopping enabled. The goal was to evaluate how well the optimized settings generalize beyond the fast 10-epoch runs used during tuning.

7.3 Results & Observations

After running both Differential Evolution (DE) and the hybrid DE+BO pipeline, the following best hyperparameters were obtained:

- **From DE:**
 - Learning Rate: 0.00574
 - Neurons in Layer 1: 32
 - Neurons in Layer 2: 58

- Batch Size: 14
- MAE: 0.0739

- **From DE + BO:**

- Learning Rate: 0.00866
- Neurons in Layer 1: 16
- Neurons in Layer 2: 16
- Batch Size: 29
- MAE: 0.0674

This resulted in an **8.8% improvement** in MAE when using the DE+BO hybrid over DE alone (which already outperforms all the other Metaheuristic algorithms tested for this report).

Final Evaluation MAE (on full test set with optimized model):

- **Accuracy:**

The DE+BO hybrid optimization consistently outperformed both the custom Differential Evolution implementation and the built-in `scipy.optimize.differential_evolution` function. Across multiple runs, the hybrid approach achieved a 5% to 15% reduction in Mean Absolute Error (MAE), depending on initialization and model configuration. This confirms the value of combining broad, global exploration (DE) with model-based refinement (BO) for more effective hyperparameter tuning.

- **Training Curves:**

The training and validation performance is visualized in the uploaded loss and MAE plots:

- **Training and validation loss** rapidly decrease during the first few epochs and then plateau at consistently low values, demonstrating fast convergence and training stability.
- **The MAE curves** for both training and validation follow a similar trend, with no visible divergence between the two, indicating the model does not overfit and maintains strong generalization to unseen data. These smooth and well-aligned curves suggest that the DE+BO-tuned model is both efficient and robust in training.

- **Prediction Quality:**

The "Actual vs. Predicted Temperature" plot shows that model predictions (red dashed line) almost perfectly match the actual temperatures (blue line) for random test samples, confirming high accuracy.

- **24-Hour Forecast:**

The 24-hour prediction plot shows stable, consistent temperature outputs. While this demonstrates model stability, the flatness suggests the model may underfit more dynamic long-term trends.

7.4 Benefits of the DE+BO Hybrid

The DE+BO hybrid optimization method offers several practical and theoretical advantages over using Differential Evolution or Bayesian Optimization alone:

- **Improved Accuracy:** Consistently achieves lower Mean Absolute Error (MAE) and more reliable predictions compared to standalone DE or other metaheuristics.
- **Faster Convergence:** Reaches optimal hyperparameter configurations in fewer evaluations and training epochs (subject to search space).
- **Robustness:** The combination of global and local search reduces the chances of getting stuck in local minima, making the optimization process more stable across different runs.
- **Better Balance between Exploration and Exploitation:** DE provides diverse global exploration across the search space, while BO efficiently exploits promising regions using a learned surrogate model.
- **Improved Overall Performance:** By combining the strengths of both methods, the hybrid approach consistently delivers better results than either optimization strategy alone.

6.5 Limitations

- **Computational Cost:** Running both Differential Evolution and Bayesian Optimization increases the total optimization time and requires more computational resources.
- **Parameter Sensitivity:** Performance depends on DE and BO settings.
- **Limited Performance Gain Over Built-in DE:** In this specific setup, the improvement of DE+BO over the built-in `scipy.optimize.differential_evolution` (over different runs under different initializations) was only around **5-15% in MAE** (a modest gain) . This is likely because the dataset had relatively few features and the problem space wasn't highly complex. The **benefits of hybrid optimization become more pronounced in larger datasets** with higher-dimensional feature spaces, where richer hyperparameter tuning has a greater impact on performance.

7. Conclusion

This project set out to systematically implement and compare a range of Artificial Neural Network (ANN) algorithms and metaheuristic optimization strategies, as outlined in the referenced research, using the ECTA weather dataset. The primary objective was to evaluate the effectiveness of different metaheuristic algorithms - namely Genetic Algorithm (GA), Differential Evolution (DE), and Particle Swarm Optimization (PSO) - for hyperparameter tuning in ANN-based weather forecasting, and to further enhance performance through a novel hybrid approach combining Differential Evolution with Bayesian Optimization (DE+BO).

Comparative Performance of Metaheuristic Algorithms

A thorough experimental evaluation was conducted using the ECTA dataset, which provided a robust testbed for short-term weather prediction. The results consistently demonstrated that:

- Differential Evolution (DE) emerged as the most effective metaheuristic algorithm for hyperparameter optimization, achieving the

lowest Mean Absolute Error (MAE). DE achieved a MAE of 0.0723, outperforming both PSO (0.1517) and GA (0.0939).

- Particle Swarm Optimization (PSO) and Genetic Algorithm (GA) lagged behind DE. Of the two, GA showed better results, while PSO, likely limited by premature convergence, displayed the highest MAE of all three metaheuristic algorithms.

The following table summarizes the comparative results for the ANN model:

Metaheuristic	MAE(Mean Absolute Error)
Differential Evolution (DE)	0.0723
Particle Swarm Optimization (PSO)	0.1571
Genetic Algorithm (GA)	0.0939
Pyhopper	0.1007
Hybrid DE + Bayesian Optimization (BO)	0.0674

Impact of the Hybrid DE+BO Approach

- Building upon the strong performance of DE, a hybrid optimization strategy was implemented, combining the global search capabilities of Differential Evolution with the local refinement strengths of Bayesian Optimization (DE+BO)
- This hybrid approach delivered an additional performance gain of approximately % over standard DE - the best-performing single metaheuristic on the ECTA dataset.

While a 8.8% improvement may appear modest in absolute terms, it is significant given the already low

error rates achieved by DE and the relatively limited size of the ECTA dataset. On smaller datasets, incremental gains become increasingly challenging due to the risk of overfitting and the limited information available for learning. The observed improvement with DE+BO thus highlights the hybrid method’s ability to finely exploit promising regions of the hyperparameter space that DE alone might miss, especially in data-constrained scenarios.

Closing Remarks

In summary, this project not only replicated and validated the findings of the reference research on metaheuristic optimization for ANN-based weather forecasting but also achieved marginal improvement through the development and validation of a hybrid DE+BO approach. This approach provides a robust and efficient method for hyperparameter tuning in neural network-based weather forecasting. It outperforms standalone DE in both accuracy and stability, as confirmed by quantitative results and visual plots. The 8.8% improvement achieved on top of the best existing method demonstrates the potential for further innovation in this space, especially when tackling the challenges posed by limited data. This method represents a significant step forward from the paper and is a key innovation in our improved forecasting system.

Contributions

Daksh Gupta	Algorithm and Report for Hybridised DE+BO, Contributed towards building custom implementation for GA.
Kabir Grewal	Implementation and Report for GA, DE and Pyhopper. Contributed in implementation for PSO.
Siddharth Bhatia	Implementation and Report for PSO. Main editor for all individual reports.
Neha Bhagwat	Researched on Bayesian Optimization and BOHB.Compilation of all the individual reports.